

EXP.NO: 5	Write a Servlet to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.
DATE:05/03/2025	

AIM:

To demonstrate the difference between the HTTP GET and POST methods by creating a form in an HTML page and handling the form submissions using two different HTTP methods in a Java Servlet.

ALGORITHM:

1. Input:

- The user fills out a web form with their name and email.
- The form will use both GET and POST methods for data submission.

2. Steps for Creating and Handling the HTTP GET and POST Requests:

1. Create an HTML Form:

- Design an HTML form with two input fields: one for the name and one for the email.
- Create two buttons to allow the user to submit the form using GET or POST methods.
- Use the action attribute to define the Servlet URL and specify the form method (either GET or POST).

2. Create the Servlet to Handle GET and POST Requests:

- Write a Java Servlet class that will handle both GET and POST requests.
- The servlet will have two methods:
 - doGet() to handle the GET requests.
 - doPost() to handle the POST requests.

- In both methods, retrieve the form data using `request.getParameter()`.
- In the `doGet()` method, process and display the data in the URL.
- In the `doPost()` method, process and display the data in the HTTP request body.

3. Handle GET Requests in the Servlet:

- In the `doGet()` method, use `request.getParameter()` to retrieve the name and email submitted via GET.
- Output the data as a response, including the name and email in the URL (visible to the user).

4. Handle POST Requests in the Servlet:

- In the `doPost()` method, use `request.getParameter()` to retrieve the name and email submitted via POST.
- Output the data as a response, displaying the name and email without exposing them in the URL (more secure).

5. Return the Output:

- The Servlet should return an HTML page displaying the name and email entered by the user.
- For the GET method, the output should be visible in the URL.
- For the POST method, the output should be visible in the response body, not the URL.

6. Deploy and Test the Servlet:

- Compile the Servlet class and deploy it to a Servlet container (like Apache Tomcat).
- Test the form using GET and POST methods to see the difference in data submission.

7. Observe and Compare GET and POST Results:

- Test the GET method: The form data is sent in the URL.
- Test the POST method: The form data is sent in the HTTP request body, not visible in the URL.

PROGRAM:

methodform.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>GET and POST Demo</title>

  <style>
    body{
      font-family: Arial;
      background:#e3f2fd;
      text-align:center;
      margin-top:50px;
    }

    .box{
      background:white;
      width:400px;
      margin:auto;
      padding:20px;
      border-radius:10px;
      box-shadow:0 0 10px gray;
      margin-bottom:20px;
    }

    input{
```

```
        width:90%;
        padding:8px;
        margin:8px;
    }

    button{
        padding:10px 20px;
        background:#0d47a1;
        color:white;
        border:none;
        cursor:pointer;
    }
</style>
</head>
<body>

<div class="box">
    <h2>GET Method - Search Student</h2>

    <form action="studentmethod" method="get">
        <input type="text" name="search" placeholder="Enter Roll
Number">
        <button type="submit">Search</button>
    </form>
</div>

<div class="box">
    <h2>POST Method - Register Student</h2>

    <form action="studentmethod" method="post">
        <input type="text" name="name" placeholder="Enter Student
Name">
        <input type="text" name="dept" placeholder="Enter Department">
```

```
        <button type="submit">Register</button>
    </form>
</div>

</body>
</html>
```

StudentMethodServlet.java:

```
import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/studentmethod")
public class StudentMethodServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String roll = request.getParameter("search");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<html><body style='font-family:Arial;text-align:center;margin-top:100px;'>");
        out.println("<h1>GET Method Used</h1>");
        out.println("<h3>Searching Student Record</h3>");
```

```
        out.println("<p>Roll Number: " + roll + "</p>");  
        out.println("</body></html>");  
    }
```

```
protected void doPost(HttpServletRequest request,  
    HttpServletResponse response)
```

```
    throws ServletException, IOException {
```

```
        String name = request.getParameter("name");  
        String dept = request.getParameter("dept");
```

```
        response.setContentType("text/html");  
        PrintWriter out = response.getWriter();
```

```
        out.println("<html><body style='font-family:Arial;text-align:center;margin-top:100px;'>");  
        out.println("<h1>POST Method Used</h1>");  
        out.println("<h3>Student Registered Successfully</h3>");  
        out.println("<p>Name: " + name + "</p>");  
        out.println("<p>Department: " + dept + "</p>");  
        out.println("</body></html>");  
    }  
}
```

Output:

The screenshot displays two distinct web forms on a light blue background. The first form, titled 'GET Method - Search Student', features a single text input field labeled 'Enter Roll Number' and a blue 'Search' button. The second form, titled 'POST Method - Register Student', contains two text input fields labeled 'Enter Student Name' and 'Enter Department', followed by a blue 'Register' button.

GET Method Used

Searching Student Record

Roll Number: 241501060

POST Method Used

Student Registered Successfully

Name: Gowtham

Department: AIML

RESULT:

A Servlet is created to demonstrate the difference between HTTP GET and POST methods by creating a form and handling requests accordingly.

